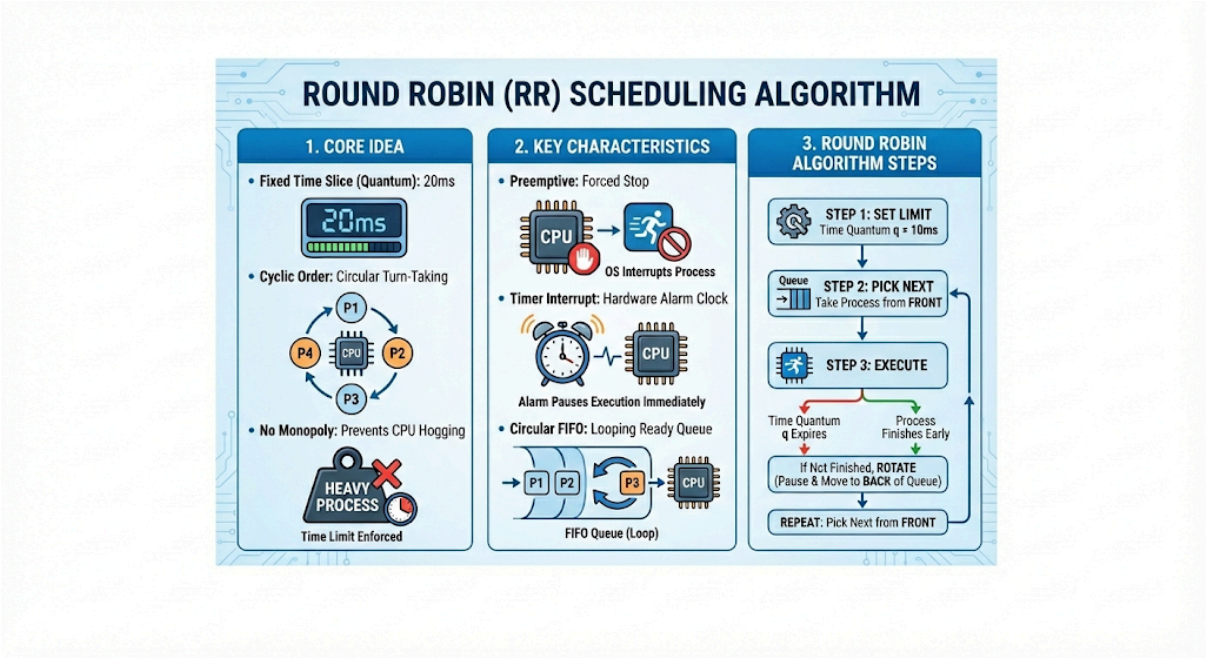
Lecture 08 NOTES – CPU Scheduling Algorithms II: Round Robin and Priority Scheduling

Round Robin (RR)

1. Core Idea

- **Fixed Time Slice (Quantum):** The "Quantum" is a strict time limit (e.g., 20 milliseconds). Every process gets exactly this much time to run.
- **Cyclic Order:** The CPU treats processes like a circle of friends taking turns. It goes around the circle one by one.
- **No Monopoly:** Because of the time limit, no single heavy process (like a video game) can hog the CPU and freeze the system.



2. Key Characteristics

- **Preemptive:** This means the OS can force a process to stop. It doesn't ask permission; when time is up, it kicks the process off.
- **Timer Interrupt:** The OS sets a hardware "alarm clock" (timer). When it rings, the current process is paused immediately.
- **Circular FIFO:** The waiting line (Ready Queue) acts like a loop. New or paused processes go to the back, and the CPU always picks from the front.

3. Round Robin Algorithm Steps

1. **Set the Limit:** Decide the time quantum q (e.g., 10ms).

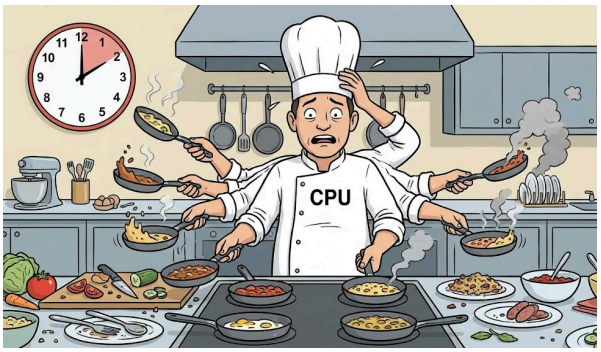
2. **Pick Next:** Take the process at the very front of the line.
3. **Execute:** Let it run. It stops when:
    - The time quantum q expires.
    - **OR** the process finishes early (whichever happens first).
4. **Rotate:** If the process isn't finished yet, pause it and send it to the **back of the line**.
5. **Repeat:** Pick the next person at the front and do it again.

Here are the revision notes for the final section, with all tags and links removed.

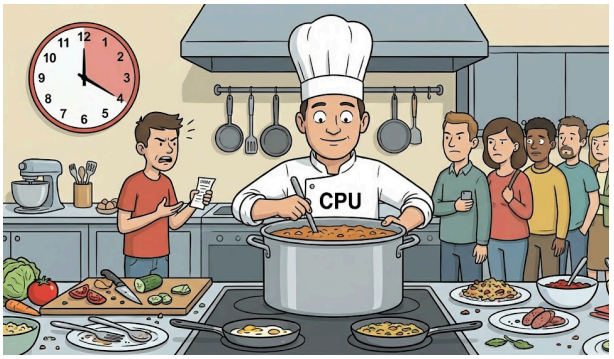
Time Quantum Concept

The **Time Quantum** (or time slice) is the specific amount of time a process is allowed to run before the OS interrupts it. Choosing the right size is critical.

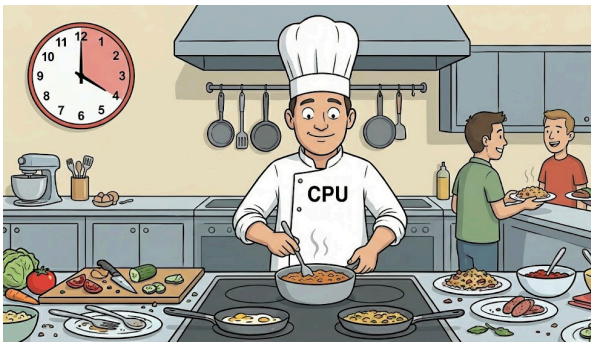
- **If Quantum is Too Small:**
    - **The Problem:** The CPU switches between tasks too frequently (e.g., running for 1ms, then switching).
    - **The Result: High Overhead**. The CPU wastes most of its energy saving and loading processes (context switching) rather than doing actual work.



- **If Quantum is Too Large:**
    - **The Problem:** The time slice is so long that processes finish before the timer ever rings.
    - **The Result: Becomes FCFS**. It behaves exactly like First-Come, First-Served. A short interactive app might get stuck waiting behind a long video render, causing **Poor Response Time**.



- **<u>Optimal Quantum</u>:**
    - **The Goal:** Find a balance. It must be large enough so that the switching cost is negligible (less than 1% of the total time), but small enough to keep the system responsive.
    - **Typical Value:** Usually between **10 to 100 milliseconds**.



Performance Metrics in Round Robin (RR)

- **<u>Response Time (Excellent):</u>**
    - This is the biggest strength of Round Robin. Since the CPU cycles through the list quickly, every process gets its *first* turn very fast.
    - Users feel the system is "snappy".
- **<u>Turnaround Time (High):</u>**
    - This is often worse than other algorithms. Because long jobs are chopped into small pieces and spread out over time, they take longer to finish completely compared to running straight through.
- **<u>Waiting Time:</u>**
    - This depends heavily on the number of processes. If many processes are in the loop, a process has to wait for everyone else's turn before running again.
- **<u>Average Waiting Time:</u>**
    - Generally higher than Shortest Job First (SJF), but Round Robin is preferred because it prevents starvation (everyone gets a fair share).

Advantages of Round Robin

- **Fair CPU Allocation:** Every process receives an equal share of the CPU time. Since the scheduler moves in a circle, no single heavy process can monopolize the processor.
- **No Starvation:** Unlike priority-based algorithms where low-priority jobs might get stuck waiting forever, Round Robin guarantees that every process will eventually get its turn.
- **Good Response Time:** This is the main benefit. Because the CPU cycles through tasks quickly, every process gets its *first* turn very fast, making apps feel responsive.
- **Best for Interactive Systems:** It is the ideal choice for personal computers, phones, and tablets where users need instant feedback when they type or click.

Disadvantages of Round Robin

- **High Context Switch Overhead:** If the time slice (quantum) is set too short, the CPU wastes a significant amount of computing power just saving and loading processes instead of executing them.
- **Poor for Batch Systems:** It is inefficient for long, calculation-heavy jobs. Because it chops large jobs into tiny pieces spread out over time, the total time to finish a job (Turnaround Time) is usually longer than simply running them one by one.
- **Not Suitable for Real-Time Systems:** Round Robin prioritizes fairness, not deadlines. It cannot guarantee that a critical task (like a car brake system) will run immediately when needed.

Round Robin in Real Systems (macOS)

- **Interactive Focus:** macOS uses Round Robin specifically for the apps you are interacting with (like Finder or Safari) to ensure they feel instant.
- **Small Slices:** It gives these apps very short time slices (quanta) so the interface updates frequently.
- **Spinning Wheel:** If an app repeatedly exceeds its time limit without responding, the OS flags it as "unresponsive" (the spinning beach ball).

Example 1: Round Robin (RR) Scheduling

Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	10
P1	1	1
P2	2	1

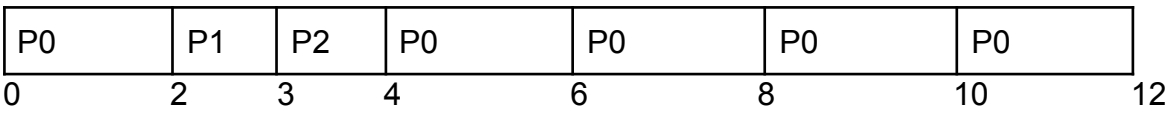
Assume:
Time Quantum (TQ) = 2 ms
Processes are scheduled using the Round Robin (RR) algorithm
Ready queue follows Circular FIFO order
Using RR scheduling:

1. Draw the Gantt Chart
2. Calculate Completion Time (CT)
3. Find Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Solution
Step 1: Scheduling Logic (Round Robin)
Each process gets 2 ms CPU time. If a process does not finish within the time quantum, it is placed at the end of the ready queue. New arriving processes are added to the ready queue immediately

Step 2: Execution Sequence (Circular FIFO Queue)
Ready Queue Order: P0 → P1 → P2 → P0 → P0 → P0 → P0

Step 3: Gantt Chart



Step 4: Completion Time (CT)

Process	Completion Time (ms)
P0	12
P1	3
P2	4

Step 5: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	12	12 − 0 = **12**
P1	1	3	3 − 1 = **2**
P2	2	4	4 − 2 = **2**

Step 6: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	12	10	12 − 10 = **2**
P1	2	1	2 − 1 = **1**
P2	2	1	2 − 1 = **1**

Step 7: Average Waiting Time (AWT)
AWT = (2+1+1)/3 = 1.33 ms

Example 2: Round Robin (RR) Scheduling

Consider the following set of processes with their Arrival Time (AT) and Burst Time (BT):

Process ID	Arrival Time (ms)	Burst Time (ms)
P0	0	5
P1	1	3
P2	2	1
P3	3	2
P4	4	3

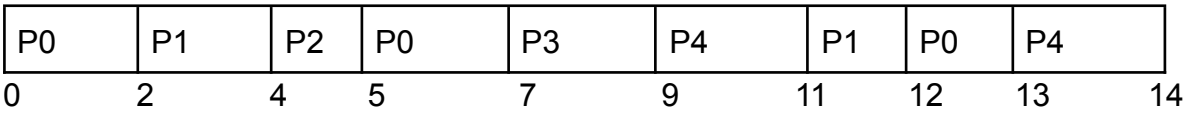
Assume:
Time Quantum (TQ) = 2 ms
Scheduling is done using the Round Robin (RR) algorithm
Ready queue follows a Circular FIFO order
Using RR scheduling:

1. Draw the Gantt Chart
2. Calculate the Completion Time (CT)
3. Find the Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Solution
Step 1: Scheduling Logic (Round Robin)
Each process gets 2 ms CPU time. If a process does not complete, it is placed at the end of the ready queue. New processes enter the queue as they arrive

Step 2: Circular FIFO Queue Order
Execution sequence: P0 → P1 → P2 → P0 → P3 → P4 → P1 → P0 → P4

Step 3: Gantt Chart



Step 4: Completion Time (CT)

Process	Completion Time (ms)
P0	13
P1	12
P2	5
P3	9
P4	14

Step 5: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	13	13 − 0 = **13**
P1	1	12	12 − 1 = **11**
P2	2	5	5 − 2 = **3**
P3	3	9	9 − 3 = **6**
P4	4	14	14 − 4 = **10**

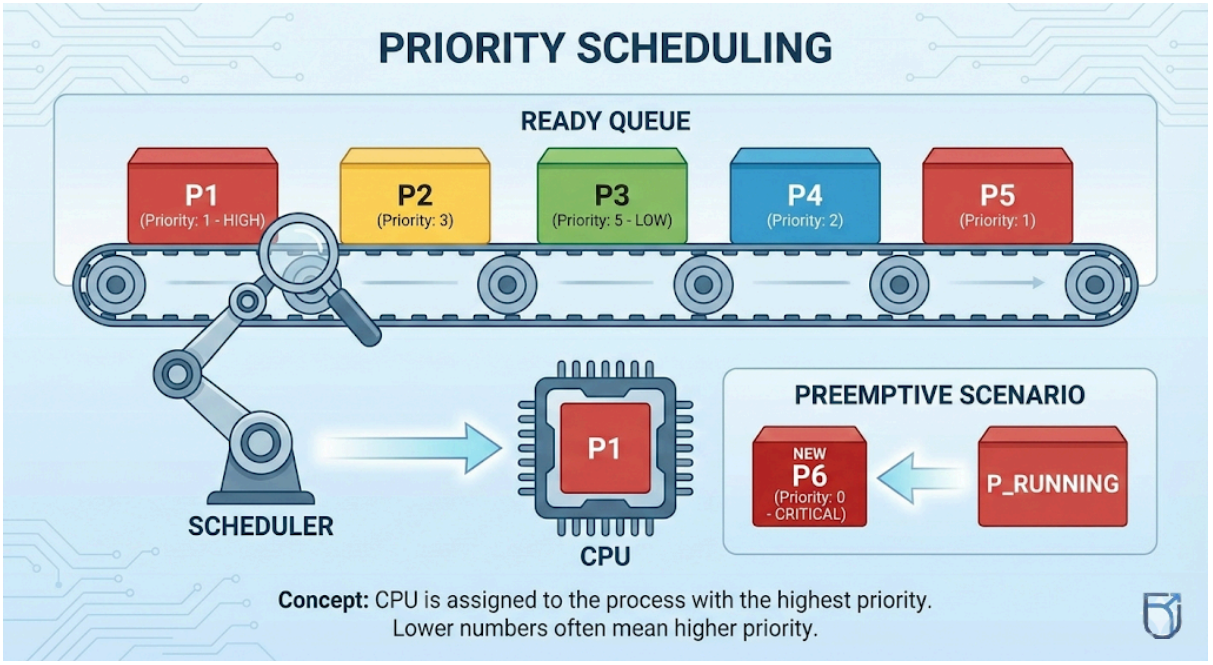
Step 6: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	13	5	**8**
P1	11	3	**8**
P2	3	1	**2**
P3	6	2	**4**
P4	10	3	**7**

Step 7: Average Waiting Time (AWT)
AWT = (8+8+2+4+7)/5 = 5.8 ms

Priority Scheduling: Core Idea

- **Ranking:** Every process is given a number (priority).
- **Rule:** The CPU is always given to the process with the highest rank.
- **Source:** Priorities can be set by the OS (internal goals) or by the user (like "High Priority" in Task Manager).



Priority Scheduling Types

- **Non-Preemptive:**
  - Once a high-priority job starts, it runs until it finishes or waits.
  - **Drawback:** Even if a *more* important job arrives, it has to wait for the current one to finish.
- **Preemptive:**
  - If a higher-priority job arrives, the OS **immediately stops** the current one to let the VIP run.
  - **Drawback:** More switching overhead, but much better responsiveness.

Priority Scheduling Algorithm Flow

1. **Assign:** Give every process a priority number.
2. **Queue:** Put processes into a line sorted by rank (highest first).
3. **Select:** Pick the one at the very front.
4. **Execute:** Run it.
   - *If Preemptive:* Watch for new, higher-priority arrivals.
5. **Repeat:** When finished or blocked, pick the next highest.

Priority Assignment Methods

How does a process get its rank? It comes from two places:

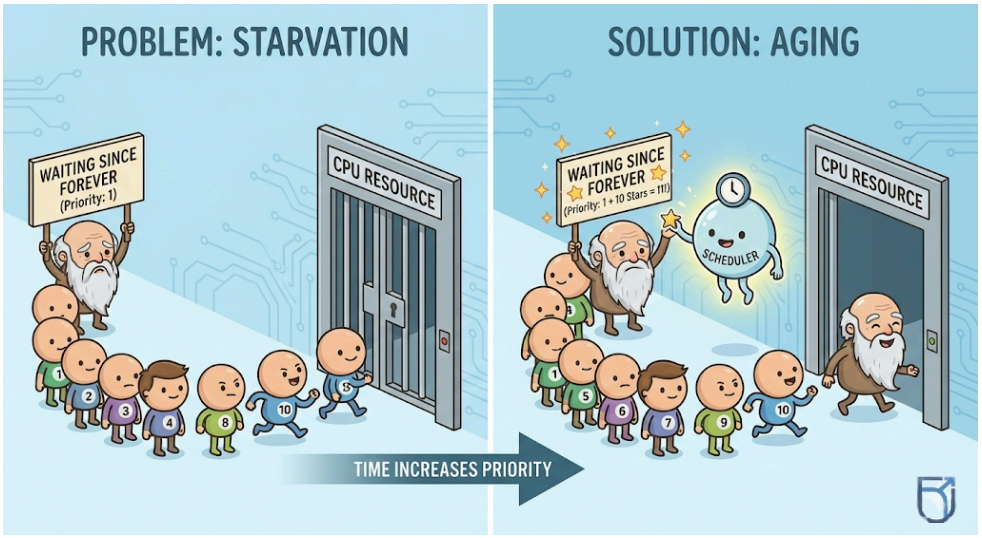
- **Internal (The System Decides):** The OS calculates this automatically using data it can measure. For example, it might give higher priority to a process that needs very little memory or one that is holding up other tasks.
- **External (Humans Decide):** These are set from the outside. This could be a user setting a specific app to "High Priority" in Task Manager, or a company policy ensuring payroll runs faster than games.

Starvation Problem

This is a major flaw in priority scheduling.

- **What happens:** If you have a low-priority task (like a background backup), it sits in the line waiting.
- **The Problem:** If high-priority tasks keep arriving, they keep cutting in front of the low-priority one. The low-priority task might wait forever and never actually run.

Aging (The Solution to Starvation)



To fix starvation, we use **Aging**.

- **How it works:** The longer a process waits in the line, the higher its priority becomes.
- **The Result:** Even if a process starts with the lowest rank, after waiting for a while, it will eventually become the highest-ranking process and get the CPU. This guarantees that nothing gets ignored forever.

Example: Non-Preemptive Priority Scheduling

Consider the following processes with their Arrival Time (AT), Burst Time (BT), and Priority (smaller priority number ⇒ higher priority):

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority

P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

Using the Non-Preemptive Priority Scheduling Algorithm:

1. Draw the Gantt Chart
2. Calculate Completion Time (CT)
3. Find Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Explanation & Correct Solution

Scheduling Rule (Non-Preemptive Priority)

CPU is allocated to the **highest-priority process** among the arrived processes. Once a process starts execution, it **runs till completion.** Lower priority value = higher priority

Step 1: Scheduling Decisions

Time 0: Only P0 has arrived → executes first

Time 4: Processes available → P1, P2, P3, P4

Highest priority = P3 (priority 1)

Time 9: Remaining → P1, P2, P4

Highest priority = P1 (priority 2)

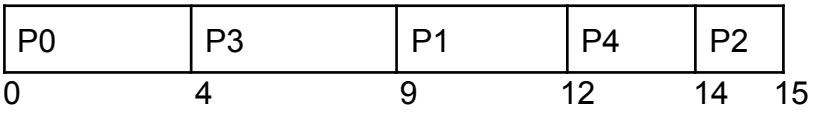
Time 12: Remaining → P2, P4

Highest priority = P4 (priority 3)

Time 14: Only P2 left

Execution Order: P0 → P3 → P1 → P4 → P2

Step 2: Gantt Chart (Corrected)



Step 3: Completion Time (CT)

Process	CT (ms)
P0	4
P3	9
P1	12
P4	14
P2	15

Step 4: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	AT	CT	TAT (ms)
P0	0	4	4
P1	1	12	11
P2	2	15	13
P3	3	9	6
P4	4	14	10

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	4	4	0
P1	11	3	8
P2	13	1	12
P3	6	5	1
P4	10	2	8

Step 6: Average Waiting Time (AWT)
AWT = (0+8+12+1+8)/5 = 5.8 ms

Question: Preemptive Priority Scheduling Algorithm

Consider the following processes with Arrival Time (AT), Burst Time (BT), and Priority (smaller number ⇒ higher priority):

Process ID	AT (ms)	BT (ms)	Priority
P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

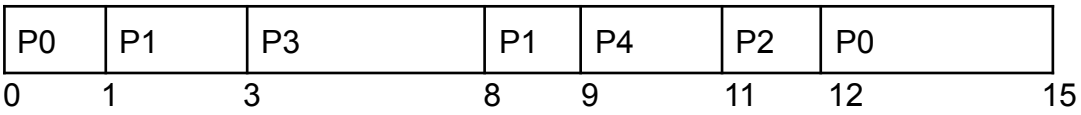
Using the Preemptive Priority Scheduling Algorithm:

1. Draw the Gantt Chart
2. Calculate Completion Time (CT)
3. Find Turnaround Time (TAT) and Waiting Time (WT)
4. Compute the Average Waiting Time (AWT)

Scheduling Logic (Preemptive Priority)
CPU is always allocated to the highest-priority process. If a higher-priority process arrives, it preempts the currently running process. Lower priority value = higher priority

Step 1: Scheduling Decisions (Timeline)
0–1: P0 executes
1–3: P1 arrives (higher priority) → preempts P0
3–8: P3 arrives (highest priority) → preempts P1
8–9: Remaining of P1 executes
9–11: P4 executes
11–12: P2 executes
12–15: Remaining of P0 executes

Step 2: Gantt Chart (Correct)



Step 3: Completion Time (CT)

Process	CT (ms)
P0	15
P1	9
P2	12
P3	8
P4	11

Step 4: Turnaround Time (TAT)
Formula:
TAT = CT − AT

Process	CT (ms)	Process	CT (ms)
P0	15	P0	15
P1	9	P1	9
P2	12	P2	12
P3	8	P3	8
P4	11	P4	11

Step 5: Waiting Time (WT)
Formula:
WT = TAT − BT

Process	TAT	BT	WT (ms)
P0	15	4	11
P1	8	3	5
P2	10	1	9
P3	5	5	0
P4	7	2	5

Step 6: Average Waiting Time (AWT)
AWT = (11+5+9+0+5)/5 = 6.0 ms

Question: Preemptive Priority Scheduling with Aging

Consider the following processes with **Arrival Time (AT)**, **Burst Time (BT)**, and **Original Priority** (**Lower priority number ⇒ higher priority**).

To prevent **starvation**, an **aging mechanism** is applied: **Aging Rule:** Every **3 ms of waiting**, increase priority by **1 level** (i.e., decrease priority number by 1).

Given Process Table:

PID	AT	BT	Original Priority
P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

1. Draw the Gantt Chart using Preemptive Priority Scheduling with Aging

2. Calculate:
    a. Completion Time (CT)
    b. Turnaround Time (TAT = CT − AT)
    c. Waiting Time (WT = TAT − BT)
3. Find the Average Waiting Time (AWT)
4. Explain how aging prevents starvation

Correct Scheduling Timeline (with Aging)

Key points during execution

- P3 has the **highest priority (1)** → once it starts, it is **never preempted**
- Aging improves priorities of waiting processes
- When priorities become equal, **FCFS among equals** is followed

Correct Gantt Chart

P0	P1	P3	P1	P2	P4	P0

0 1 3 8 9 10 12 15

Correct Final Calculations

PID	AT	BT	Priority Evolution	CT	TAT = CT−AT	WT = TAT−BT
P0	0	4	5 → 4 → 3 → 2	15	15	11
P1	1	3	2 → 1	9	8	5
P2	2	1	4 → 3 → 2	10	8	7
P3	3	5	1	8	5	0
P4	4	2	3 → 2 → 1	12	8	6

Average Waiting Time (AWT)

AWT=(11+5+7+0+6)/5 = 5.8 ms

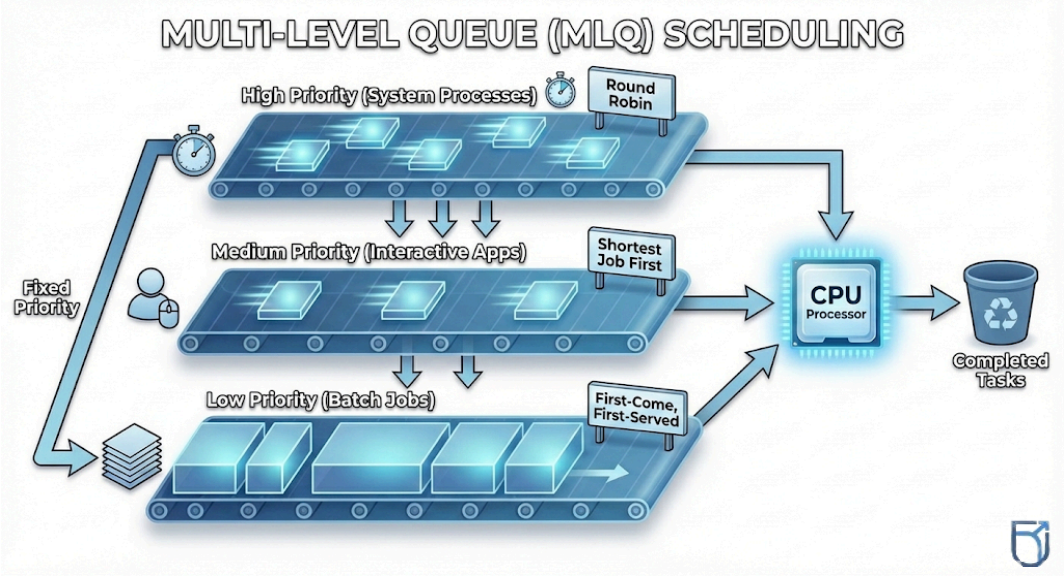
macOS Priority Scheduling (Quality of Service)

macOS classifies tasks into "lanes" based on how much the user notices them. It's like a VIP system for your apps.

- **User Interactive (Highest):**
    - **What it is:** The "Main Thread." Anything involved in drawing the screen or animations.

    - **Goal:** Must finish immediately (within 1 frame) or the screen freezes.
- **User Initiated:**
    - **What it is:** Tasks the user explicitly asked for, like opening a document or clicking a button.
    - **Goal:** The user is waiting for the result, so it shouldn't be delayed.
- **Default:**
    - **What it is:** The standard priority for general application work.
- **Utility:**
    - **What it is:** Long-running tasks the user is aware of, like a progress bar for downloading a file or importing photos.
- **Background (Lowest):**
    - **What it is:** Invisible maintenance tasks like indexing files (Spotlight) or backups.
    - **Goal:** Only runs when the CPU is effectively idle so it doesn't slow down your work.

Multi-Level Queue (MLQ) Scheduling



Think of this as a bank with different windows for different types of customers.

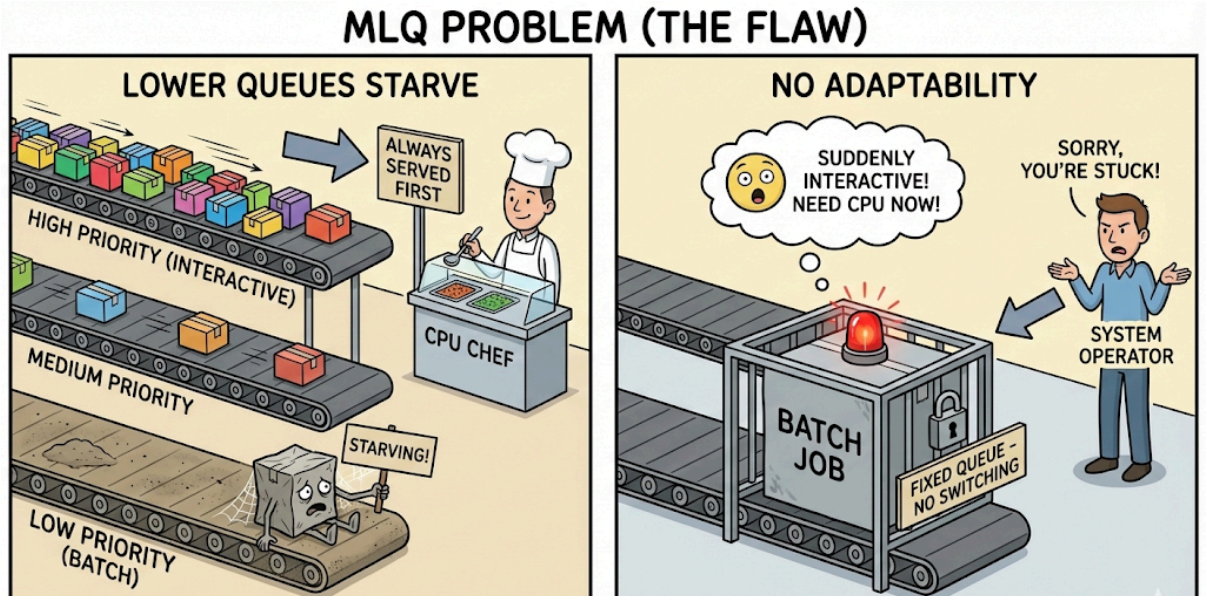
- **Multiple Fixed Queues:** The Ready Queue is split into separate lines (e.g., one for System tasks, one for Student tasks).
- **Fixed Assignment:** When a process enters the system, it is assigned to **one** queue and stays there forever. It cannot move.
- **Independent Rules:** Each queue can have its own rules.
    - *Example:* The "Interactive Queue" might use **Round Robin** (for speed), while the "Batch Queue" uses **FCFS** (for simplicity).
- **Rigid Priority:** The CPU looks at the queues in order. It will *only* touch the lower queues if the top queues are completely empty.

MLQ Queue Structure Example

The computer usually sets up the lines in this order of importance:

1. **Queue 1: System / Real-Time (Top Priority)**
    - **Who:** Critical OS tasks (like memory management).
    - **Rule:** Must run immediately.
2. **Queue 2: Interactive (Medium Priority)**
    - **Who:** Your web browser, games, editors.
    - **Rule:** Uses **Round Robin** so everything feels snappy.
3. **Queue 3: Batch (Low Priority)**
    - **Who:** Email sync, weather updates, file downloads.
    - **Rule:** Uses **FCFS**. These only run when you stop using the computer actively.

MLQ Problem (The Flaw)



- **Lower Queues Starve:** Since the CPU always finishes the top queues first, the bottom queues (batch jobs) might never run if there is constant high-priority work.
- **No Adaptability:** Once a process is in a queue, it is stuck there. If a "batch" job suddenly becomes "interactive," the system cannot adjust to help it.

Multi-Level Feedback Queue (MLFQ)

- **The Fix:** This is an upgraded version of MLQ where processes **can move** between queues.

- **Dynamic Priority:** The system watches how a process behaves and adjusts its rank automatically.
    - **CPU-Heavy (Greedy):** If a process hogs the CPU, it gets **demoted** to a lower priority queue.
    - **I/O-Bound (Interactive):** If a process releases the CPU quickly (waiting for input), it gets **promoted** to keep it snappy.

MLFQ Core Rules

1. **Start High:** Every new process enters at the **highest priority queue**.

2. **Punish Hogs:** If a process uses its entire time slice (quantum) without stopping, it is pushed down to a lower queue.
3. **Reward Niceness:** If a process pauses early (e.g., waiting for a key press) or waits too long in a low queue, it is bumped up.
4. **Aging:** To fix starvation, the OS will eventually boost any process that has been stuck at the bottom for too long.

MLFQ in macOS

- **Real-World Use:** Your Mac uses this constantly to manage hundreds of apps.
- **Variable Time Quanta:**
    - **Top Queue (Interactive):** Very short time slices (e.g., 10ms) for quick response.
    - **Bottom Queue (Background):** Long time slices (e.g., 100ms) for efficiency.
- **Balance:** This ensures your active window feels fast, while big downloads still finish in the background without freezing the system.